Unordered Set | Ordered Set

Unordered Set

Definition

An unordered set is a collection of unique elements with no particular order. It is implemented using a hash table.

Key Properties

a) Unique elements (like set).
b) Not sorted. (order depends on hash values)
c) Faster average time for operations.

Ordered Set

Definition

An ordered set is a sorted ordered collection of unique elements. It is implemented as a BST tree. (typically a Red-Black Tree)

Key Properties

a) Unique elements (like set).
b) Automatically sorted in increasing ordered.
c) Slower than Unordered Set for search/insert/erase.

Unordered Set | Ordered Set | Common Operations

1) Declaration + Allocation

Unordered Set (hash-based, no sorting)

```
1) unordered_set<int> mySet1;
2) unordered_set<int> mySet2 = {1, 2, 3};
```

Ordered Set (BST-based, sorted)

```
1) set<int> mySet1;
2) set<int> mySet2 = {1, 2, 3};
```

Note

a) In Unordered Set elements are in random order.

b) In Ordered Set elements are always sorted (by default in ascending order).

for descending ordered the signature as given below.

```
set<int, greater<int>> mySet1
```

Unordered Set | Ordered Set | Common Operations

2) Adding an element

Same Signature for Both

```
1) mySet.insert(element);
```

3) Adding all elements

Same Signature for Both

```
1) mySet1.insert(mySet2.begin(), mySet2.end());
```

4) Remove an element

Same Signature for Both

```
1) mySet.erase(element);
```

5) Getting the Size

Same Signature for Both

```
1) mySet.size();
```

Unordered Set | Ordered Set | Common Operations

6) Two Sets are equal or not

Same Signature for Both

```
1) mySet1 == mySet2;
```

7) set1 is a subset of set2

Unordered set

```
Check manually
```

Ordered set

```
1) includes(set2.begin(),set2.end(), set1.begin(), set1.end())
```

8) Checking Existence

Same Signature for Both

```
1) set1.find(element) != set1.end();   → iterator
2) set.count(element);                 → 1/0 size_t/int
```

Unordered Set | Ordered Set | Common Operations

9) Sum of Set

Same Signature for Both

```
1) int sum = accumulate(s.begin(), s.end(), 0)
```

10) Max of Set

Same Signature for Both

```
1) int max = *max_element(mySet.begin(), mySet.end());
```

11) Min of Set

Same Signature for Both

```
1) int min = *min_element(mySet.begin(), mySet.end());
```

11) Looping Set

Same Signature for Both

```
1) for (int val : mySet) { cout << val ; }
```

12) Union of Sets

Same for Both

In Place

```
1) set1.insert(set2.begin(), set2.end());
```

In New variable

```
1) unordered_set<int> newSet = set1;
2) newSet.insert(set2.begin(), set2.end());
```

```
1) set<int> newSet = set1;
2) newSet.insert(set2.begin(), set2.end());
```

Note

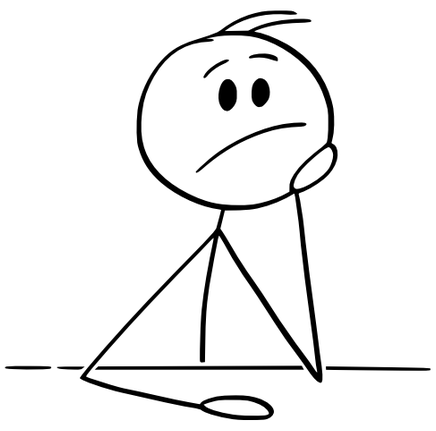
a) In an ordered set, we can also use the inbuilt `set_union()` function to find the union of two sets.

12) Union of Sets

Note

a) In an ordered set, we can also use the inbuilt `set_union()` function to find the union of two sets.

```
1) set<int> A = {1, 2, 3, 4};
2) set<int> B = {3, 4, 5, 6};
3) set<int> set1;
4) set_union(A.begin(),A.end(),B.begin(),B.end(),
                inserter(set1, set1.begin()));
```



What is `inserter()`?

inserter(container, position) is a helper function from `<iterator>` that creates an **output iterator** which calls `container.insert()` for each value assigned to it.

13) Intersection of Set

Unordered set (hash-based, no sorting)

Manually

```
1) unordered_set<int> newSet;
2) for (auto num : Set1){
3)  if (Set2.count(num))
4)    {newSet.insert(num);} }
```

Ordered set (BST-based, sorted)

```
1) set<int> A = {1, 2, 3, 4};
2) set<int> B = {3, 4, 5, 6};
3) set<int> set1;
4) set_intersection(A.begin(),A.end(),B.begin(),B.end(),
                    inserter(set1, set1.begin()));
```

14) Difference of Set

Unordered set (hash-based, no sorting)

Manually

```
1) unordered_set<int> diffSet;
2) for (auto num : Set1){
3)  if (!Set2.count(num))
4)   {diffSet.insert(num);} }
```

Ordered set (BST-based, sorted)

```
1) set<int> A = {1, 2, 3, 4};
2) set<int> B = {3, 4, 5, 6};
3) set<int> set1;
4) set_difference(A.begin(),A.end(),B.begin(),B.end(),
                  inserter(set1, set1.begin())); → A-B
```

14) Difference of Set

Ordered set (BST-based, sorted)

```
1) set<int> A = {1, 2, 3, 4};
2) set<int> B = {3, 4, 5, 6};
3) set<int> set1;
4) set_difference(A.begin(),A.end(),B.begin(),B.end(),
                  inserter(set1, set1.begin())); → A-B
```

Note

Explanation:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

set_difference(A, B) returns elements that are in A but not in B.

A - B = {1, 2}

15) Sorting a Set

Unordered set (hash-based, no sorting)

Ascending Sorting of a Set (Convert to vector first)

```
1) set<int> S1 = {1, 12, 3, 14};
2) vector<int> new_vec(S1.begin(), S1.end()); →copy elements
3) sort(v.begin(), v.end()); → {1, 3, 12, 14}
```

Descending Sorting of a Set (Convert to vector first)

```
1) set<int> S1 = {1, 12, 3, 14};
2) vector<int> new_vec(S1.begin(), S1.end()); →copy elements
3) sort(v.begin(), v.end(), greater<int>()); → {14,12,3,1}
```

Time Complexity Comparison

Unordered set

Uses a hash table.

1) Best case:

O(1) for insert, erase, find (when hash distribution is good).

2) Worst case:

O(N) if all keys hash into the same bucket or rehashing occurs.

Ordered set

Uses a balanced BST (typically a Red-Black Tree).

1) All times are deterministic — O(log N) for insert, erase, and find.

2) There’s no difference between best, average, and worst case because balancing keeps the tree height ≈ log N.

Time Complexity Comparison

Operation	Ordered_set Time-Complexity	unordered_set Time-Complexity	
	Best and Worst Case	Best Case	Worst Case
Insert single element	O(log n)	O(1)	O(n)
Insert all elements	O(n log (m))	O(m)	O(n*m)
Erase an element	O(log n)	O(1)	O(n)
Equality check (==)	O(n)	O(n)	O(n)
Subset check (includes)	O(n + m)	O(n)	O(n)
Check existence (find)	O(log n)	O(1)	O(n)
Count of element	O(n)	O(1)	O(n)
Sum/Max/min element	O(n)	O(n)	O(n)
Union (insert all elements)	O(n log (m))	O(m)	O(n*m)
Intersection	O(n + m)	O(n)	O(n*m)
Difference	O(n + m)	O(n)	O(n*m)

1	**Unique Numbers**
Problem Statement	Given an array of N integers. Print the unique elements from the array in increasing order. Also print the count of unique elements. See sample below.
I/P Format	First line of input contains the value of N. The second line contains N space separated integers.
O/P Format	Two lines of output. First line will have space separated integers and the second line will have a single integer.
Constraint	`1 <= N <= 10^5` `-10^6 <= arr[i] <= 10^6`
Sample Input	11 10 2 3 4 2 5 6 4 2 3 6
Sample Output	2 3 4 5 6 10 6

2	**Count Commons**
Problem Statement	Given two strings S1 and S2, having lengths N and M respectively. Print the count of characters from S1, which are also present in S2. Duplicate characters in S1 should be counted only once, and small and capital cases should be considered different.
I/P Format	Three lines of input. First line has space separated values of N and M.
O/P Format	A single integer.
Constraint	`1 <= N, M <= 10^6`

Sample Input	`13 16`
	`Quickbrownfox`
	`Learnfromlearnys`

Sample Output	`4`

3	**Mutual Uncommon**
Problem Statement	You are given two arrays having M and N integers respectively. Let P be the number of elements in the first array but not in the second array. Let Q be the number of elements in the second array but not in the first array. Print the value of P * Q.
I/P Format	The first line contains space separated values of M and N. The next two lines contain M and N space separated integers.
O/P Format	Single integer.
Constraint	`1 <= N, M <= 10^6`
Sample Input	5 6 2 3 1 4 12 10 14 4 12 2 7
Sample Output	6